

Aress Frontend Project - Technical Documentation

Table of Contents

1. [Project Overview](#)
 2. [Architecture & Technology Stack](#)
 3. [Project Structure](#)
 4. [Design System Documentation](#)
 5. [Applications Overview](#)
 6. [Development Setup](#)
 7. [Development Workflow](#)
 8. [API Integration](#)
 9. [Testing Strategy](#)
 10. [Deployment](#)
-

Project Overview

The **Aress Frontend Project** is a sophisticated enterprise-grade monorepo built with Nx, designed to support multiple frontend applications with a shared design system. The project emphasizes code reusability, maintainability, and developer experience through modern tooling and best practices.

Key Features

- **Monorepo Architecture:** Nx-powered workspace for efficient code sharing
- **Comprehensive Design System:** 60+ reusable UI components
- **Multiple Applications:** B2B and B2C applications with shared infrastructure
- **Modern Tooling:** TypeScript, Tailwind CSS, Storybook, Vitest
- **API-First Approach:** OpenAPI/Swagger integration with code generation

- **Production Ready:** Docker support, CI/CD workflows, and quality gates
-

Architecture & Technology Stack

Core Technologies

- **Framework:** React 18+ with Next.js 15
- **Language:** TypeScript 5+
- **Styling:** Tailwind CSS with custom design tokens
- **Build System:** Nx 21.3.0 with Vite/Webpack
- **Package Manager:** pnpm (workspace support)

Development Tools

- **Component Development:** Storybook 9.0
- **Testing:** Vitest + Jest + Playwright
- **Code Quality:** ESLint, Prettier, Husky, Commitlint
- **Icons:** Iconify with Lucide icon set
- **Documentation:** Storybook with auto-docs

Infrastructure

- **Containerization:** Docker with multi-stage builds
 - **Deployment:** Vercel integration
 - **API Integration:** OpenAPI code generation
 - **State Management:** React Query (TanStack Query)
-

Project Structure

```
adress-frontend/
├── apps/                # Applications
│   ├── fe-app/          # Main B2B application (Next.js)
│   ├── b2c-app/         # B2C application (Next.js)
│   ├── fe-app-e2e/      # E2E tests for fe-app
│   └── renderer/        # Rendering service
├── libs/               # Shared libraries
└── design-system/      # UI component library
```

```
|   | openapi/           # Generated API clients
|   | shared/           # Shared utilities and hooks
|   | scripts/          # Build and utility scripts
|   | shared/           # Global shared resources
|   | .storybook/       # Global Storybook configuration
|   | docker-compose.yml # Development environment
|   | nx.json           # Nx workspace configuration
|   | package.json       # Root package configuration
```

Key Directories Explained

`/apps` - Applications

Contains all deployable applications, each with their own Next.js configuration, routing, and application-specific components.

`/libs` - Shared Libraries

- **design-system**: Core UI component library with Storybook documentation
- **openapi**: Auto-generated API clients and React Query hooks
- **shared**: Common utilities, hooks, and helper functions

`/scripts` - Automation

Contains build scripts, including the Tailwind CSS design system color generator.

Design System Documentation

Overview

The design system is located in `libs/design-system` and provides a comprehensive set of 60+ reusable UI components built with React, TypeScript, and Tailwind CSS.

Component Architecture

Each component follows a consistent structure:

```
ComponentName/
├── ComponentName.tsx      # Main component implementation
├── ComponentName.types.ts # TypeScript interfaces (if
multiple)
```

└─ ComponentName.constants.ts	# Component constants (if needed)
└─ ComponentName.stories.tsx	# Storybook stories
└─ ComponentName.test.tsx	# Unit tests
└─ index.ts	# Exports

Core Components

1. Button Component

Location: `libs/design-system/src/lib/components/Button/`

Purpose: Primary interactive element with multiple variants and states.

Key Props:

```
interface ButtonProps extends
  React.ButtonHTMLAttributes<HTMLButtonElement> {
  isLoading?: boolean;           // Loading state with spinner
  iconLeft?: IconProps;         // Left-side icon
  iconRight?: IconProps;        // Right-side icon
  size?: 'sm' | 'md';           // Size variants
  mode?: 'primary' | 'secondary' | 'text' | 'underline'; // Visual
                                modes
  align?: 'center' | 'right';    // Content alignment
  theme?: 'brand' | 'error' | 'success' | 'neutral'; // Color
                                themes
}
```

Usage Example:

```
import { Button } from '@aress/design-system';

<Button
  mode="primary"
  theme="brand"
  size="md"
  iconLeft={{ name: 'plus', size: 'sm' }}
  isLoading={false}
>
  Add Item
</Button>
```

2. Icon Component

Location: `libs/design-system/src/lib/components/Icon/`

Purpose: Unified icon system supporting both Lucide icons and custom SVGs.

Key Props:

```
interface IconProps {  
  name: IconName;           // Icon identifier  
  size?: IconSize;          // Size variant  
}
```

Features: - Automatic detection of custom vs. Lucide icons - Memoized for performance - Consistent sizing system - Support for custom SVG icons

Usage Example:

```
import { Icon } from '@aress/design-system';  
  
<Icon name="user" size="md" />  
<Icon name="CustomLogo" size="lg" />
```

3. Form Components

TextField

Purpose: Input field with validation and styling support.

Features: - Built-in validation states - Label and helper text support - Icon integration - Responsive design

Checkbox & Radio

Purpose: Selection controls with consistent styling.

Features: - Custom styling matching design system - Proper accessibility support - Group management utilities

4. Layout Components

Header

Purpose: Application header with navigation and user controls.

Breadcrumb

Purpose: Navigation breadcrumb trail.

Accordion

Purpose: Collapsible content sections.

5. Data Display Components

GeneralTable

Purpose: Data table with sorting, filtering, and pagination.

DataList

Purpose: Structured data presentation.

Badge

Purpose: Status indicators and labels.

6. Feedback Components

Dialog/Modal

Purpose: Modal dialogs and overlays.

Notification

Purpose: Toast notifications and alerts.

ProgressBar

Purpose: Progress indication.

7. Media Components

VideoPlayer

Purpose: Custom video player with controls.

Features: - Custom controls - Keyboard navigation - Responsive design - Multiple format support

8. Navigation Components

Tabs

Purpose: Tab-based navigation.

Pagination

Purpose: Page navigation controls.

ContextMenu

Purpose: Right-click context menus.

Design Tokens

The design system uses Tailwind CSS with custom design tokens defined in the configuration. Colors are automatically generated from CSS variables using a custom script.

Color System: - Brand colors (primary, secondary) - Semantic colors (success, error, warning, info) - Neutral colors (grays, backgrounds) - Theme support (light/dark modes)

Typography: - Consistent font scales - Line height ratios - Font weight variations

Spacing: - 8px base unit system - Consistent margin/padding scales

Storybook Integration

Each component includes Storybook stories for: - **Default state:** Basic component usage - **Variants:** All available props and combinations - **Interactive examples:** Real-world usage scenarios - **Documentation:** Auto-generated prop tables

Running Storybook:

```
nx storybook design-system
```

Hooks and Utilities

Custom Hooks

- **UseVideo:** Video player state management
- **CustomToast:** Toast notification system
- **DayPicker:** Date selection utilities

Utility Functions

- **classNames:** Conditional CSS class management
- **debounce:** Function debouncing
- **number-utils:** Number formatting and calculations
- **time:** Time manipulation utilities

Applications Overview

1. FE-App (B2B Application)

Location: apps/fe-app/

Purpose: Main business-to-business application for financial data and investment management.

Architecture

- **Framework:** Next.js 15 with App Router
- **Routing Structure:**

```
app/
├── (auth)/                # Authentication pages
├── (dashboard)/           # Main dashboard area
│   ├── (nofooter)/       # Pages without footer
│   └── (withfooter)/      # Pages with footer
├── (internal_services)/   # Internal service pages
├── (layout)/              # Layout components
├── api/                   # API routes
└── components/           # App-specific components
```

Key Features

- **Authentication System:** Login, registration, password recovery
- **Investment Dashboard:** Portfolio management and analytics
- **Fund Management:** Investment fund tracking and reporting
- **Data Visualization:** Charts, graphs, and financial metrics
- **Report Generation:** PDF and Excel export capabilities
- **User Profile Management:** Settings and preferences

Technology Integration

- **API Integration:** OpenAPI-generated clients
- **State Management:** React Query for server state
- **Styling:** Tailwind CSS with design system components
- **Authentication:** Middleware-based route protection
- **File Handling:** Upload and processing capabilities

Configuration Files

- `next.config.ts`: Next.js configuration with custom webpack setup
- `middleware.ts`: Authentication and routing middleware

- `tailwind.config.js`: Tailwind configuration extending design system

2. B2C-App (Consumer Application)

Location: `apps/b2c-app/`

Purpose: Consumer-facing application for public users and retail investors.

Architecture

- **Framework:** Next.js 15 with App Router
- **Routing Structure:**

```
app/
├── (auth)/           # Public authentication
├── (layout)/         # Layout components
├── (no-footer)/      # Pages without footer
├── (withfooter)/     # Pages with footer
├── components/       # B2C-specific components
└── lib/             # B2C utilities
```

Key Features

- **Public Dashboard:** Market overview and public data
- **User Registration:** Simplified onboarding flow
- **Educational Content:** Investment guides and resources
- **Market Data:** Real-time financial information
- **Responsive Design:** Mobile-first approach

Differences from FE-App

- Simplified user interface
- Public data focus
- Reduced feature complexity
- Consumer-oriented workflows

3. Renderer Application

Location: `apps/renderer/`

Purpose: Specialized rendering service for generating reports, charts, and visual content.

Features

- Server-side rendering capabilities
- Chart and graph generation
- PDF report creation
- Image processing

Shared Application Features

Common Dependencies

Both main applications share: - Design system components - API client libraries
- Shared utilities and hooks - Common styling approach - Similar build and deployment processes

Environment Configuration

- Development: Local development with hot reload
 - Staging: Pre-production testing environment
 - Production: Optimized builds with CDN integration
-

Development Setup

Prerequisites

- **Node.js**: Version 18+ (LTS recommended)
- **pnpm**: Version 8+ (package manager)
- **Nx CLI**: Global installation recommended

Installation Steps

1. Install Global Dependencies:

```
npm install -g nx pnpm
```

2. Clone and Setup:

```
git clone <repository-url>  
cd aress-frontend
```

```
pnpm install
```

3. Environment Configuration:

```
# Copy environment templates  
cp apps/fe-app/.env.local.example apps/fe-app/.env.local  
# Configure your environment variables
```

Development Commands

Application Development

```
# Run main B2B application  
nx dev fe-app  
  
# Run B2C application  
nx dev b2c-app  
  
# Run both applications  
nx run-many --target=dev --projects=fe-app,b2c-app
```

Design System Development

```
# Run Storybook for component development  
nx storybook design-system  
  
# Build design system  
nx build design-system
```

Testing

```
# Run all tests  
nx test  
  
# Run specific project tests  
nx test design-system  
nx test fe-app  
  
# Run E2E tests  
nx e2e fe-app-e2e  
  
# Run tests with UI  
pnpm test:ui
```

Code Quality

```
# Lint all projects
nx lint

# Format code
nx format

# Strict linting (CI mode)
pnpm lint:strict
```

Docker Development

Using Docker Compose

```
# Start development environment
docker-compose up

# Build specific service
docker-compose build fe-app
```

Individual Docker Builds

```
# Build FE-App
docker build -f Dockerfile.fe-app -t aress-fe-app .

# Build B2C-App
docker build -f Dockerfile.b2c-app -t aress-b2c-app .
```

Development Workflow

Branch Naming Convention

```
{type}/{card-id}/{description}
```

Types: - `feat`: New features - `fix`: Bug fixes - `docs`: Documentation updates - `refactor`: Code refactoring - `test`: Test additions/updates

Example:

```
git checkout -b feat/HaAUwSZv/add-user-profile-component
```

Commit Convention

The project uses Conventional Commits with Commitlint:

```
# Feature commit
git commit -m "feat(design-system): add new Button component variants"

# Bug fix commit
git commit -m "fix(fe-app): resolve authentication redirect issue"

# Documentation commit
git commit -m "docs(readme): update installation instructions"
```

Pull Request Process

1. **Create Feature Branch:** Branch from `main`
2. **Implement Changes:** Follow coding standards
3. **Write Tests:** Ensure adequate test coverage
4. **Update Documentation:** Update relevant docs
5. **Submit PR:** Use provided PR template
6. **Code Review:** Address reviewer feedback
7. **Merge:** Squash and merge after approval

Code Quality Gates

Pre-commit Hooks (Husky)

- **Lint:** ESLint validation
- **Format:** Prettier formatting
- **Type Check:** TypeScript compilation
- **Test:** Affected tests execution

CI/CD Pipeline

- **Build Validation:** All projects build successfully
- **Test Execution:** Unit and integration tests
- **E2E Testing:** Critical user journeys

- **Security Scanning:** Dependency vulnerability checks
 - **Performance Testing:** Bundle size analysis
-

API Integration

OpenAPI/Swagger Integration

The project uses a sophisticated API integration strategy with automatic code generation:

Workflow

1. **API Specification:** OpenAPI/Swagger definitions
2. **Code Generation:** Automatic TypeScript client generation
3. **React Query Integration:** Generated hooks for data fetching
4. **Type Safety:** Full TypeScript support

Commands

```
# Merge multiple Swagger files
pnpm merge:swagger

# Generate TypeScript API client
pnpm generate:sdk

# Generate React Query hooks
pnpm generate:query

# Complete API update workflow
pnpm output:swagger
```

Generated Structure

```
libs/openapi/src/
├── queries/           # React Query hooks
├── requests/         # API request functions
└── types/            # TypeScript interfaces
```

Usage Example

```
import { useGetUserProfile } from '@aress/openapi';
```

```
function UserProfile() {  
  const { data, isLoading, error } = useGetUserProfile();  
  
  if (isLoading) return <div>Loading...</div>;  
  if (error) return <div>Error: {error.message}</div>;  
  
  return <div>Welcome, {data?.name}</div>;  
}
```

Testing Strategy

Testing Pyramid

Unit Tests (Vitest)

- **Component Testing:** Individual component behavior
- **Utility Testing:** Helper function validation
- **Hook Testing:** Custom hook functionality

```
# Run unit tests  
nx test design-system  
nx test fe-app
```

Integration Tests (Jest)

- **API Integration:** Service layer testing
- **Component Integration:** Multi-component workflows
- **State Management:** Complex state scenarios

E2E Tests (Playwright)

- **User Journeys:** Critical application flows
- **Cross-browser Testing:** Chrome, Firefox, Safari
- **Mobile Testing:** Responsive behavior validation

```
# Run E2E tests  
nx e2e fe-app-e2e
```

Test Organization

Component Tests

```
// Button.test.tsx
import { render, screen } from '@testing-library/react';
import { Button } from './Button';

describe('Button Component', () => {
  it('renders with correct text', () => {
    render(<Button>Click me</Button>);
    expect(screen.getByText('Click me')).toBeInTheDocument();
  });

  it('handles loading state', () => {
    render(<Button isLoading>Loading</Button>);
    expect(screen.getByRole('button')).toBeDisabled();
  });
});
```

Storybook Testing

- **Visual Regression:** Chromatic integration
 - **Interaction Testing:** User interaction scenarios
 - **Accessibility Testing:** A11y compliance validation
-

Deployment

Build Process

Production Builds

```
# Build all applications
nx build

# Build specific application
nx build fe-app
nx build b2c-app

# Build design system
nx build design-system
```

Docker Deployment

```
# Multi-stage Docker builds  
docker build -f Dockerfile.fe-app -t aress-fe-app:latest .  
docker build -f Dockerfile.b2c-app -t aress-b2c-app:latest .
```

Deployment Targets

Vercel Integration

- **Automatic Deployments:** Git-based deployments
- **Preview Deployments:** PR-based previews
- **Environment Variables:** Secure configuration management

Container Orchestration

- **Docker Compose:** Local development
- **Kubernetes:** Production orchestration
- **Load Balancing:** High availability setup

Environment Configuration

Development

- Hot reload enabled
- Source maps included
- Debug logging active

Staging

- Production-like environment
- Performance monitoring
- User acceptance testing

Production

- Optimized bundles
 - CDN integration
 - Error tracking
 - Performance monitoring
-

Best Practices & Guidelines

Component Development

1. **Single Responsibility:** Each component has one clear purpose
2. **Prop Interface:** Well-defined TypeScript interfaces
3. **Default Props:** Sensible defaults for optional props
4. **Error Boundaries:** Graceful error handling
5. **Accessibility:** WCAG 2.1 compliance

Code Organization

1. **File Naming:** PascalCase for components, camelCase for utilities
2. **Import Order:** External, internal, relative imports
3. **Export Strategy:** Named exports preferred over default
4. **Type Definitions:** Co-located with components when simple

Performance Optimization

1. **Code Splitting:** Route-based and component-based splitting
2. **Lazy Loading:** Dynamic imports for heavy components
3. **Memoization:** React.memo for expensive components
4. **Bundle Analysis:** Regular bundle size monitoring

Security Considerations

1. **Input Validation:** Client and server-side validation
 2. **XSS Prevention:** Proper data sanitization
 3. **CSRF Protection:** Token-based protection
 4. **Dependency Scanning:** Regular security audits
-

Troubleshooting

Common Issues

Build Errors

- **TypeScript Errors:** Check type definitions and imports
- **Missing Dependencies:** Verify package.json and node_modules
- **Path Resolution:** Check tsconfig.json path mappings

Development Issues

- **Hot Reload:** Restart development server
- **Port Conflicts:** Check for running processes
- **Cache Issues:** Clear Nx cache with `nx reset`

Storybook Issues

- **Component Loading:** Verify component exports
- **Addon Conflicts:** Check Storybook configuration
- **Build Failures:** Clear Storybook cache

Getting Help

1. **Documentation:** Check component Storybook stories
 2. **Code Examples:** Review existing implementations
 3. **Team Communication:** Use established communication channels
 4. **Issue Tracking:** Create detailed bug reports
-

Contributing

Code Standards

- Follow established ESLint and Prettier configurations
- Write comprehensive tests for new features
- Update documentation for public APIs
- Follow conventional commit messages

Review Process

- All changes require peer review
- Automated tests must pass

- Documentation must be updated
- Performance impact must be considered

Release Process

- Semantic versioning for design system
 - Changelog maintenance
 - Migration guides for breaking changes
 - Backward compatibility considerations
-

Appendix

Useful Commands Reference

```
# Development
nx dev fe-app           # Start FE app
nx dev b2c-app          # Start B2C app
nx storybook design-system # Start Storybook

# Testing
nx test                 # Run all tests
nx e2e fe-app-e2e      # Run E2E tests
pnpm test:ui           # Test with UI

# Building
nx build                # Build all
nx build fe-app         # Build FE app

# Code Quality
nx lint                 # Lint all
nx format               # Format all
pnpm lint:strict        # Strict linting

# API Integration
pnpm output:swagger     # Update API clients
```

Project Contacts

- **Technical Lead:** [Contact Information]
- **Design System Maintainer:** [Contact Information]
- **DevOps/Infrastructure:** [Contact Information]

External Resources

- [Nx Documentation](#)
- [Next.js Documentation](#)
- [Tailwind CSS Documentation](#)
- [Storybook Documentation](#)

This documentation is maintained by the development team and updated regularly. For the most current information, please refer to the project repository and Storybook documentation.